Конашинская Ольга Александровна 090303-Пиб-о24

Лабораторная работа №5

Вариант: № 10

Тема: Получить практические навыки использования функциональных возможностей языка Python для обработки коллекций данных и построения последовательностей.

Постановка задачи: Необходимо разработать консольное приложение обработки набора объектов выбранной предметной области.

В качестве исходных данных необходимо использовать **объекты классов**, то есть желательно продолжить работу с предметной областью лабораторной работы №3 или выбрать новый вариант.

Приложение должно демонстрировать использование функциональных возможностей Python.

Программа должна содержать:

1. не менее одного базового класса;
2. не менее двух производных классов;
3. коллекцию объектов;
4. функции обработки объектов;
5. минимум две функции высшего порядка;
6. минимум две lambda-функции;
7. минимум один map();
8. минимум один filter();
9. минимум один вызов sorted(..., key=...);
10. минимум один comprehension;
11. минимум один генератор;
12. минимум одну проверку с any() или all();
13. собственный итератор или генератор;
14. обработку исключительных ситуаций.

Краткие теоретические сведения:

Функции как объекты и высший порядок

- Функции — это объекты первого класса: В Python функции можно записывать в переменные, передавать как аргументы в другие функции,

возвращать из функций и сохранять в структуры данных (например, списки).

- Функции высшего порядка: Это функции, которые принимают другие функции в качестве параметров или возвращают их (например, apply_operation()).
- Анонимные функции (lambda): Используются для создания коротких функций в одну строку, чаще всего прямо внутри вызовов других функций.

Встроенные инструменты функционального программирования

- map(): Поэлементно применяет заданную функцию к коллекции и возвращает итератор.
- filter(): Фильтрует элементы коллекции с помощью функции-предиката, оставляя только те, что возвращают True.
- reduce(): Импортируется из functools. Последовательно сжимает коллекцию в одно единственное значение (например, для вычисления суммы или произведения), но не должна заменять собой более понятные встроенные sum(), min() или max().
- sorted(): Позволяет настраивать логику сортировки сложных объектов через аргумент key.
- any() и all(): Быстрые проверки коллекций на истинность. any() возвращает True, если истинен хотя бы один элемент, а all() — если истинны все.

Абстракции для создания коллекций (Comprehensions)

- Синтаксический сахар: С помощью емкого синтаксиса в одну строку можно генерировать списки (List comprehension), множества (Set comprehension) и словари (Dictionary comprehension), в том числе используя условия фильтрации if.

Генераторы и итераторы (Ленивые вычисления)

- Генераторы: Функции, которые поочередно отдают значения с помощью ключевого слова yield, не сохраняя всю последовательность в памяти сразу. Они вычисляются «лениво» — только в момент запроса, что сильно экономит оперативную память.

- Генераторные выражения: Записываются в круглых скобках (x for x in...) по аналогии с генераторами списков, но возвращают ленивый объект-генератор.
- Итераторы и их протокол: Объект является итератором, если в нем реализованы методы __iter__() и __next__(). При вызове функции next() он отдает следующий элемент, а когда элементы заканчиваются, выбрасывает исключение StopIteration (которое цикл for обрабатывает автоматически).
- Связь понятий: Генератор — это просто более легкий и удобный способ создать итератор без ручного написания методов __iter__ и __next__.

Если вам нужно глубже разобраться в материале, скажите, и я могу:

- Составить для вас практический тест (квиз) по этим темам для проверки знаний.
- Написать дополнительные примеры для самых сложных концептов (например, reduce или кастомные итераторы).
- Объяснить подробнее разницу между итерируемым объектом и итератором.

Исходный код программы:

main.py

```
from models import MeatDish, VeggieDish
from operations import (
    sort_by_price,
    filter_by_caloric,
    get_all_prices,
    get_average_caloric,
    has_veggie_dishes,
)
from generators import dish_generator_by_category

menu = [
    MeatDish("Борщ с говядиной", "Супы", 250, 350),
    VeggieDish("Салат Цезарь Вег", "Салаты", 180, 150),
    MeatDish("Стейк", "Горячее", 600, 500),
    VeggieDish("Овощи гриль", "Горячее", 220, 120)
]

def main():
    while True:
        print("===== МЕНЮ =====",
              "1 - Показать все объекты",
```

```
              "2 - Отфильтровать объекты",
              "3 - Преобразовать данные",
              "4 - Отсортировать объекты",
              "5 - Выполнить статистический анализ",
              "6 - Запустить генератор",
              "7 - Проверить условие",
              "0 - Выход", sep="\n")

        choice = input("Выберите пункт меню: ")

        if choice == "1":
            print("Список всех блюд в меню")
            for item in menu:
                print(
                    f"Название: {item.name} | Категория: {item.category} |
Цена: {item.price} руб. | Калорийность: {item.caloric} ккал")

        elif choice == "2":
            print("Фильтрация объектов по калорийности")
            try:
                max_cal = float(input("Введите максимальную калорийность: "))
                filtered_dishes = filter_by_caloric(menu, max_cal)
                if filtered_dishes:
                    for item in filtered_dishes:
                        print(f" Подходит: {item.name} ({item.caloric}
ккал)")
                else:
                    print("Блюд с такой или меньшей калорийностью не
найдено.")
            except ValueError:
                print("Ошибка: Введено не число!")

        elif choice == "3":
            print("Преобразование данных (Получение списка всех цен)")
            prices_list = get_all_prices(menu)
            print(f"Список цен всех позиций: {prices_list}")

        elif choice == "4":
            print("Сортировка объектов по цене")
            sorted_menu = sort_by_price(menu)
            for item in sorted_menu:
                print(f"{item.name} — {item.price} руб.")

        elif choice == "5":
            print("Статистический анализ (Средняя калорийность)")
            avg_caloric = get_average_caloric(menu)
            print(f"Средняя калорийность всех блюд в меню: {avg_caloric:.2f}
ккал")

        elif choice == "6":
            print("Запуск генератора по категории")
            cat = input("Введите интересующую категорию (например: Горячее,
Супы, Салаты): ")
            my_gen = dish_generator_by_category(menu, cat)

            print(f"Результаты генератора для категории '{cat}':")
            found = False
            for item in my_gen:
                print(f" Найдено через yield: {item.name}")
                found = True
            if not found:
                print("В этой категории пока нет блюд.")

        elif choice == "7":
```

```
            print("Проверка условия (Наличие вегетарианских блюд) ---")
            if has_veggie_dishes(menu):
                print(" Результат проверки: Да, в меню присутствуют
вегетарианские блюда.")
            else:
                print(" Результат проверки: Нет, вегетарианских блюд в меню
не обнаружено.")

        elif choice == "0":
            print("Программа завершена!")
            break
        else:
            print("Некорректный ввод.")


if __name__ == "__main__":
    main()
```

generators.py

```
def dish_generator_by_category(data, target_category):
    for line in data:
        if line.category.lower() == target_category.lower():
            yield line
```

models.py

```
class Dish:
    def __init__(self, name, category, price, caloric):
        self.name = name
        self.category = category
        self.price = float(price)
        self.caloric = float(caloric)

class VeggieDish(Dish):
    def __init__(self, name, category, price, caloric):
        super().__init__(name, category, price, caloric)
        self.is_veggie = True

class MeatDish(Dish):
    def __init__(self, name, category, price, caloric):
        super().__init__(name, category, price, caloric)
        self.is_veggie = False
```

operations.py

```
from models import VeggieDish

def final_price(names, data):
    price = 0
    for line in data:
        for name in names:
            if name == line.name:
                price += float(line.price)
    return price
```

```
def filter_by_caloric(data, max_calories):
    filtered_data = filter(lambda line: line.caloric <= max_calories, data)
    return list(filtered_data)

def get_all_prices(data):
    prices = map(lambda line: line.price, data)
    return list(prices)

def sort_by_price(data):
    return sorted(data, key=lambda line: line.price)

def get_average_caloric(data):
    try:
        all_calories = [line.caloric for line in data]
        average = sum(all_calories) / len(all_calories)
        return average
    except ZeroDivisionError:
        print("Ошибка: Список меню пуст, невозможно посчитать среднюю
калорийность!")
        return 0

def has_veggie_dishes(data):
    check = any(isinstance(line, VeggieDish) for line in data)
    return check

def process_menu(data, action_function):
    return action_function(data)

def custom_filter(data, condition_function):
    results = []
    for line in data:
        if condition_function(line):
            results.append(line)
    return results
```

```
===== МЕНЮ =====
1 - Показать все объекты
2 - Отфильтровать объекты
3 - Преобразовать данные
4 - Отсортировать объекты
5 - Выполнить статистический анализ
6 - Запустить генератор
7 - Проверить условие
0 - Выход
Выберите пункт меню: 1

--- Список всех блюд в меню ---
Название: Борщ с говядиной | Категория: Супы | Цена: 250.0 руб. | Калорийность: 350.0 ккал
Название: Салат Цезарь Вег | Категория: Салаты | Цена: 180.0 руб. | Калорийность: 150.0 ккал
Название: Стейк | Категория: Горячее | Цена: 600.0 руб. | Калорийность: 500.0 ккал
Название: Овощи гриль | Категория: Горячее | Цена: 220.0 руб. | Калорийность: 120.0 ккал
```

```
Выберите пункт меню: 2
Фильтрация объектов по калорийности
Введите максимальную калорийность: 600
 Подходит: Борщ с говядиной (350.0 ккал)
 Подходит: Салат Цезарь Вег (150.0 ккал)
 Подходит: Стейк (500.0 ккал)
 Подходит: Овощи гриль (120.0 ккал)
```

```
Выберите пункт меню: 3
Преобразование данных (Получение списка всех цен)
Список цен всех позиций: [250.0, 180.0, 600.0, 220.0]
```

```
Выберите пункт меню: 4
Сортировка объектов по цене
Салат Цезарь Вег — 180.0 руб.
Овощи гриль — 220.0 руб.
Борщ с говядиной — 250.0 руб.
Стейк — 600.0 руб.
```

```
Выберите пункт меню: 5
Статистический анализ (Средняя калорийность)
Средняя калорийность всех блюд в меню: 280.00 ккал
```

```
Выберите пункт меню: 6
Запуск генератора по категории
Введите интересующую категорию (например: Горячее, Супы, Салаты): Горячее
Результаты генератора для категории 'Горячее':
 Найдено через yield: Стейк
 Найдено через yield: Овощи гриль
```

```
Выберите пункт меню: 7
Проверка условия (Наличие вегетарианских блюд) ---
 Результат проверки: Да, в меню присутствуют вегетарианские блюда.
```